

Section 1 : Introduction au Stream Processing et Kafka Streams

Cette section introduit les concepts fondamentaux du traitement de flux en temps réel et positionne Kafka Streams dans l'écosystème Kafka. Les étudiants devront expliquer la différence entre traitement par batch et stream processing, présenter l'architecture de Kafka Streams (topologies, KStream vs KTable).

Un exemple simple de transformation de flux (filtrage, mapping) doit être inclus avec du code.

Section 2 : Opérations Stateless dans Kafka Streams

Cette section approfondit les transformations sans état (stateless) dans Kafka Streams : filter, map, flatMap, branch, merge, etc. Les étudiants doivent créer un exemple de topologies simples montrant comment transformer un flux d'événements doctorat-events (par exemple : filtrer les soutenances validées, transformer le format JSON, enrichir avec des métadonnées).

L'objectif est de comprendre le chaînage d'opérations et la notion de topologie DAG (Directed Acyclic Graph).

Section 3 : Opérations Stateful et State Stores

Cette section couvre les transformations avec état : agrégations (count, reduce, aggregate), jointures (stream-stream, stream-table, table-table), et windowing (tumbling, hopping, sliding windows). Les étudiants expliqueront comment Kafka Streams maintient l'état localement via RocksDB, gère la fault-tolerance avec des changelog topics, et permettent des requêtes interactives. Un exemple de comptage de soutenances par mois ou par laboratoire doit illustrer ces concepts.

Section 4 : Introduction à Spring Batch - Architecture et concepts

Cette section présente Spring Batch comme framework de traitement par lots : architecture (JobRepository, JobLauncher, Job, Step), cycle de vie d'un job, notion de chunk-oriented processing, et gestion des transactions. Les étudiants devront expliquer quand utiliser du batch processing vs stream processing, présenter les composants (ItemReader, ItemProcessor, ItemWriter), et montrer un exemple simple de job qui lit des données CSV et les écrit en base.

Section 5 : Jobs, Steps, Tasklets et Orchestration dans Spring Batch

Cette section approfondit la création de jobs complexes : enchaînement de steps, gestion des transitions conditionnelles (décisions), parallélisation (partitioning, multi-threading) erreurs et restart. Les étudiants illustreront avec un cas d'usage concret (ex: traitement mensuel de rapports de soutenances) et montreront comment orchestrer plusieurs étapes métier avec rollback en cas d'échec.

Section 6 : Spring Batch alimenté par Kafka (Event-Driven Batch)

Cette section montre l'intégration entre Spring Batch et Kafka : utiliser Kafka comme source de données pour un ItemReader personnalisé, consommer des événements en batch depuis un topic, et déclencher des jobs Spring Batch suite à la réception d'événements spécifiques. Les étudiants expliqueront des cas d'usage hybrides (ex: agréger des événements temps réel accumulés dans Kafka pour générer des rapports quotidiens) et compareront avec le stream processing pur.

Section 7 : Pattern CQRS (Command Query Responsibility Segregation)

Cette section introduit le pattern CQRS : séparation du modèle d'écriture (Command) et de lecture (Query), motivations architecturales (scalabilité indépendante, optimisation des modèles), et implémentation dans un contexte microservices. Les étudiants devront présenter un exemple avec une base d'écriture (PostgreSQL pour les commandes) et une base de lecture optimisée (MongoDB ou Elasticsearch pour les queries), synchronisées via des événements Kafka. Les avantages et complexités du pattern doivent être discutés.

Section 8 : Pattern Event Sourcing - Concepts de base

Cette section couvre l'Event Sourcing : stocker l'état comme séquence d'événements immuables plutôt que l'état actuel, reconstruction de l'état par replay, snapshots pour optimisation. Les étudiants expliqueront la différence avec une architecture CRUD classique, montreront comment Kafka sert naturellement de Event Store, et présenteront un exemple (ex: historique complet d'un doctorat reconstitué depuis les événements). Les liens avec CQRS et les défis (migrations de schéma d'événements) doivent être abordés.

Section 9 : Intégration de Kafka dans Spring Boot avec Spring Kafka

Pour cette section: Il faut créer des microservices Producer et Consumer avec Spring Boot et Spring Kafka. Les étudiants devront montrer la configuration (application.yml), l'utilisation de `@KafkaListener` et `KafkaTemplate`, la sérialisation/désérialisation JSON (la gestion des erreurs et retry ça sera un plus) et les concepts de `containerFactory`. Un exemple complet end-to-end (un microservice publie des événements + un autre microservice les consomme) avec code source doit être fourni.